

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ
ΕΡΓΑΣΤΗΡΙΟ ΥΠΟΛΟΓΙΣΤΙΚΩΝ ΣΥΣΤΗΜΑΤΩΝ
http://www.cslab.ece.ntua.gr

Εργαστήριο Λειτουργικών Συστημάτων

7ο εξάμηνο, Ακαδημαϊκή περίοδος 2026–2027

Εργαστηριακή Άσκηση 2 - Lunix-TNG

Στήσιμο Περιβάλλοντος Εργασίας

Εργαστήριο Υπολογιστικών Συστημάτων Ε.Μ.Π.
os-lab@lists.cslab.ece.ntua.gr

2 Νοεμβρίου 2026

1 Εισαγωγή

Ο οδηγός περιγράφει τι χρειάζεται για να ξεκινήσετε να δουλεύετε την Άσκηση 2. Υποθέτουμε ότι έχετε ήδη ένα VM με πρόσφατο Ubuntu ή Debian. Οποιαδήποτε άλλη σύγχρονη διανομή είναι επίσης εντάξει, ωστόσο εμείς έχουμε δοκιμάσει τα βήματα σε αυτές τις δύο διανομές, κάποιες εντολές πιθανώς να απαιτούν μικρές προσαρμογές (π.χ., εντολές εγκατάστασης πακέτων με `apt`). Τα βήματα που πρέπει να κάνετε για να ξεκινήσετε είναι τρία:

1. Σύνδεση μιας σειριακής θύρας του VM με τον server των αισθητήρων.
2. Μεταγλώττιση και εγκατάσταση στο VM της έκδοσης 7.3 του πυρήνα Linux.
3. Λήψη και μεταγλώττιση του βοηθητικού κώδικα, φόρτωση του module και έλεγχος ότι ο driver λαμβάνει δεδομένα απο τους αισθητήρες.

2 Σύνδεση του VM με το δίκτυο αισθητήρων

Ο σταθμός βάσης είναι συνδεδεμένος με `USB` σε ένα μηχάνημα που βρίσκεται στο server room του CSLab. Τα δεδομένα των αισθητήρων φτάνουν στον σταθμό βάσης σε δυαδική μορφή. Τα δεδομένα που λαμβάνονται απο τους αισθητήρες προωθούνται μέσω TCP απο το `lunix.cslab.ece.ntua.gr`, στη θύρα 49152. Απο τη μεριάς σας θα πρέπει να ρυθμίσετε τον hypervisor ώστε μια εικονική σειριακή θύρα του VM να συνδέεται ως client σε αυτή τη διεύθυνση/πόρτα. Μέσα στο VM η θύρα εμφανίζεται ως κανονική σειριακή συσκευή (συνήθως `/dev/ttyS0`), και ο πυρήνας δεν ξέρει ότι πίσω της υπάρχει δίκτυο.

2.1 Έλεγχος ότι ο server είναι προσβάσιμος

Αρχικά επιβεβαιώστε οτι ο server είναι προσβάσιμος με `nc`:

```
nc -v lunix.cslab.ece.ntua.gr 49152
```

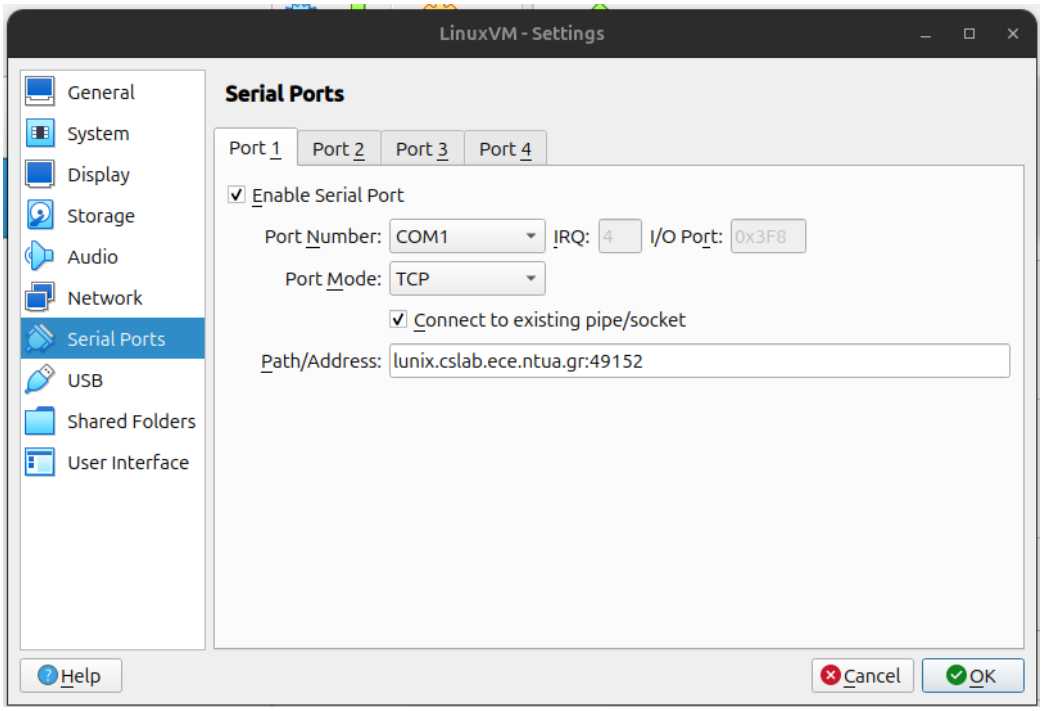
Πρέπει να δείτε γραμμές με δεκαεξαδικά bytes. Αν η σύνδεση δεν ανοίγει, το πρόβλημα είναι στο δίκτυο και όχι στο VM, οπότε λύστε το πρώτα.

2.2 Προσθήκη σειριακής θύρας στο VM

Τερματίστε τη λειτουργία του VM και ακολουθήστε τις οδηγίες για τον hypervisor που χρησιμοποιείτε.

2.2.1 VirtualBox

Στην περίπτωση που χρησιμοποιείτε το VirtualBox για το VM σας θα πρέπει να πάτε στις ρυθμίσεις του VM και να ενεργοποιήσετε τη σειριακή πόρτα, όπως φαίνεται στην παρακάτω εικόνα:

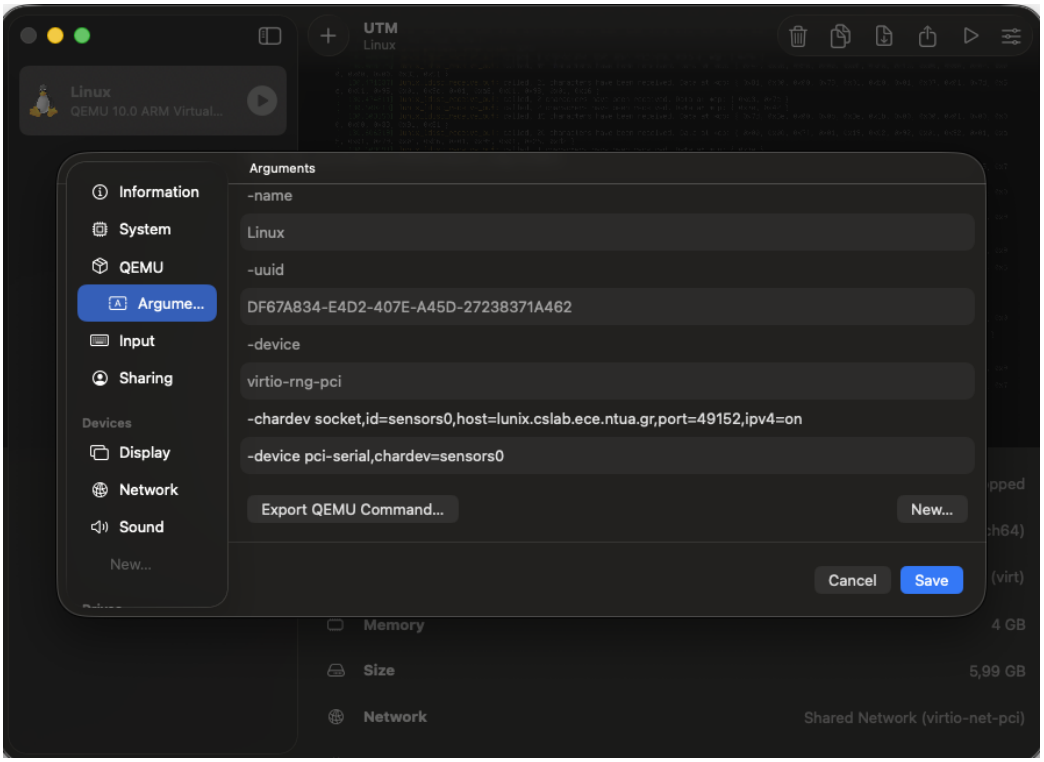


Σχήμα 1: Ρύθμιση σειριακής θύρας στο VirtualBox.

ΠΡΟΣΟΧΗ: Το VirtualBox συνδέεται στον server μόνο όταν ξεκινά το VM. Αν κάνετε reboot από μέσα (ή Reset από το μενού του VirtualBox), η σύνδεση χάνεται και δεν αποκαθίσταται, οπότε στο `ttyS0` δεν φτάνουν πια δεδομένα. Σε αυτή την περίπτωση θα πρέπει να τερματίσετε πλήρως το VM και να το ξεκινήσετε ξανά.

2.2.2 UTM

Στην περίπτωση που χρησιμοποιείτε UTM για το VM σας θα πρέπει να πάτε στις ρυθμίσεις και να προσθέσετε δύο επιπλέον arguments στο QEMU, όπως φαίνεται στην παρακάτω εικόνα:



Σχήμα 2: Ρύθμιση σειριακής θύρας στο UTM.

2.3 Έλεγχος μέσα στο VM

Ξεκινήστε το VM και βρείτε ποια συσκευή είναι η σειριακή θύρα:

```
sudo dmesg | grep 'ttyS'
```

Θα πρέπει να δείτε μια γραμμή όπως:

```
00:04: ttyS0 at I/O 0x3f8 (irq = 4, base_baud = 115200) is a 16550A
```

Στη συνέχεια βάλτε τη θύρα σε raw mode, ώστε το tty layer να μην αλλοιώνει τα bytes και να μην τα στέλνει πίσω στον server ως echo. Αν όλα έχουν πάει καλά θα πρέπει να μπορείτε να διαβάσετε δεδομένα, αν και σε δυαδική μορφή:

```
sudo stty -F /dev/ttyS0 raw -echo
sudo cat /dev/ttyS0
```

3 Μεταγλώττιση του πυρήνα Linux 7.3

Για την άσκηση θα χρησιμοποιήσετε τον πυρήνα Linux 7.3, τον οποίο θα μεταγλωττίσετε και θα εγκαταστήσετε μόνοι σας στο VM. Η ενότητα περιγράφει πώς το κάνετε ξεκινώντας από μια φρέσκια εγκατάσταση Debian (12 ή 13) ή Ubuntu (24.04 ή νεότερη), πώς κάνετε εκκίνηση με τον νέο πυρήνα και πώς ελέγχετε ότι μπορείτε να μεταγλωττίσετε ένα kernel module γι' αυτόν. Όπου μια εντολή απαιτεί δικαιώματα διαχειριστή, εκτελείται με `sudo`. Στο debian η εντολή `sudo` μπορεί να μην είναι ήδη εγκατεστημένη, θα πρέπει να την εγκαταστήσετε και να προσθέτε τον χρήστη σας στο sudo group.

3.1 Πριν ξεκινήσετε

Καλό θα ήταν να πάρετε ένα snapshot του VM πριν μεταγλωττίσετε τον πυρήνα 7.3, ώστε να μπορείτε εύκολα να το επαναφέρετε σε περίπτωση που κάτι πάει στραβά. Επίσης, προτείνουμε να έχετε έναν φάκελο διαμοιραζόμενο απο το μηχάνημά σας μέσα στο VM και να έχετε εκεί τα αρχεία της άσκησης, ώστε αν κάτι πάει στραβά με το VM να μην τα χάσετε.

Ελάχιστοι πόροι που προτείνουμε για το VM:

- Δίσκος: 20GB. Με το `localmodconfig` ο πηγαίος κώδικας και τα αρχεία της μεταγλώττισης χρειάζονται περίπου 5 έως 8GB.
- Μνήμη: 4GB ή παραπάνω.
- CPUs: όσες μπορείτε να διαθέσετε. Ο χρόνος μεταγλώττισης μειώνεται σχεδόν αναλογικά.

ΠΡΟΣΟΧΗ: Ο πυρήνας που θα φτιάξετε δεν είναι υπογεγραμμένος από τη διανομή. Αν το VM εκκινεί με UEFI και Secure Boot ενεργό, απενεργοποιήστε το Secure Boot από τις ρυθμίσεις firmware του VM, αλλιώς ο νέος πυρήνας δεν θα φορτώσει.

Σημειώστε ποιον πυρήνα έχετε τώρα, για να μπορείτε να συγκρίνετε στο τέλος:

```
uname -r
```

3.2 Εγκατάσταση των εργαλείων μεταγλώττισης

```
sudo apt update
sudo apt install -y gcc make gawk bc bison flex libssl-dev \
                    libelf-dev dwarves libdw-dev
```

3.3 Λήψη του πηγαίου κώδικα

Κατεβάστε το επίσημο tarball από το kernel.org και αποσυμπιέστε το σε έναν φάκελο στο home σας. Η μεταγλώττιση γίνεται ως απλός χρήστης.

```
mkdir -p ~/kernel
cd ~/kernel
wget https://cdn.kernel.org/pub/linux/kernel/v7.x/linux-7.3.tar.xz
tar xf linux-7.3.tar.xz
cd linux-7.3
```

Από εδώ και πέρα όλες οι εντολές της ενότητας τρέχουν μέσα στον φάκελο `~/kernel/linux-7.3`, εκτός αν αναφέρεται κάτι άλλο.

3.4 Ρύθμιση του πυρήνα

Ο πυρήνας ρυθμίζεται μέσω του αρχείου `.config`, που περιέχει μερικές χιλιάδες επιλογές `CONFIG_*`. Για την άσκηση είναι αρκετό να στηριχθείτε στο config του πυρήνα που ήδη τρέχει, που είναι γνωστό ότι εκκινεί στο μηχάνημά σας.

3.4.1 Αντιγραφή του τρέχοντος config

Αρχικά ξεκινάτε αντιγράφοντας το configuration αρχείο του πυρήνα που τρέχει τώρα στο σύστημά σας:

```
cp /boot/config-$(uname -r) .config
```

3.4.2 Περιορισμός στα modules που χρησιμοποιεί το σύστημά σας

Το config της διανομής χτίζει χιλιάδες drivers για υλικό που δεν έχετε, και η μεταγλώττιση μπορεί να κρατήσει ώρες. Το `localmodconfig` κρατά ως modules μόνο όσα είναι φορτωμένα αυτή τη στιγμή (όπως τα δείχνει η `lsmod`) και απενεργοποιεί τα υπόλοιπα.

```
yes "" | make localmodconfig
```

Το `yes ""` απαντά με την προεπιλογή σε κάθε ερώτηση για νέες επιλογές.

ΠΡΟΣΟΧΗ: Ό,τι δεν είναι φορτωμένο εκείνη τη στιγμή δεν θα χτιστεί. Αν θέλετε υποστήριξη για κάποια συσκευή (π.χ. ένα USB stick ή ένα σειριακό adapter USB), συνδέστε την και βεβαιωθείτε ότι εμφανίζεται στην `lsmod` πριν τρέξετε την εντολή.

3.4.3 Απαραίτητες διορθώσεις για config Debian/Ubuntu

Τα config των διανομών αναφέρονται σε πιστοποιητικά που υπάρχουν μόνο στο δικό τους build σύστημα (π.χ., `debian/canonical-certs.pem`). Αν δεν τα αφαιρέσετε, η μεταγλώττιση αποτυγχάνει. Επίσης, καλό είναι να απενεργοποιήσετε τα debug symbols, που πολλαπλασιάζουν τον χρόνο και τον χώρο στον δίσκο. Τέλος, δώστε ένα δικό σας επίθεμα στην έκδοση, ώστε ο πυρήνας σας να ξεχωρίζει από αυτόν της διανομής.

```
scripts/config --set-str SYSTEM_TRUSTED_KEYS ""
scripts/config --set-str SYSTEM_REVOCATION_KEYS ""
scripts/config --disable DEBUG_INFO_DWARF_TOOLCHAIN_DEFAULT
scripts/config --disable DEBUG_INFO_DWARF4
scripts/config --disable DEBUG_INFO_DWARF5
scripts/config --enable  DEBUG_INFO_NONE
scripts/config --set-str LOCALVERSION "-oslab"
make olddefconfig
```

Το `make olddefconfig` δίνει προεπιλεγμένες τιμές σε όσες επιλογές της 7.3 δεν υπήρχαν στο παλιό config και επιλύει τις εξαρτήσεις μεταξύ επιλογών. Το `scripts/config` απλώς γράφει στο `.config` χωρίς να ελέγχει εξαρτήσεις, οπότε μετά από κάθε αλλαγή στο `.config` εκτός menuconfig τρέχετε πάντα `make olddefconfig`.

Ελέγξτε ότι οι αλλαγές «κράτησαν», ότι δεν έμειναν επιλογές χωρίς τιμή:

```
grep -E 'SYSTEM_TRUSTED_KEYS|SYSTEM_REVOCATION_KEYS|DEBUG_INFO_NONE|
    LOCALVERSION=' .config
make listnewconfig
```

Η `make listnewconfig` δεν πρέπει να τυπώνει τίποτα. Αν τυπώνει επιλογές, το make θα σας ρωτήσει γι' αυτές κατά τη μεταγλώττιση: τρέξτε ξανά `make olddefconfig`.

3.4.4 Προαιρετικά: εξερεύνηση των ρυθμίσεων

```
sudo apt install -y libncurses-dev
make menuconfig
```

Με το «/» αναζητάτε μια επιλογή με το όνομά της. Με το «?» βλέπετε την περιγραφή της και από ποιες άλλες επιλογές εξαρτάται. Αξίζει να το ανοίξετε μια φορά για να δείτε πώς είναι οργανωμένος ο πυρήνας, αλλά για την άσκηση δεν απαιτείται καμία αλλαγή.

3.5 Μεταγλώττιση

```
time make -j$(nproc)
```

Το `-j$(nproc)` τρέχει τόσες παράλληλες εργασίες όσες είναι οι CPUs του VM. Η μεταγλώττιση παράγει:

- την εικόνα του πυρήνα, `arch/x86/boot/bzImage` (για x86_64),
- τα modules (αρχεία `.ko`) μέσα στο δέντρο του κώδικα,
- το `vmlinux` και το `System.map`, που χρησιμεύουν στην αποσφαλμάτωση.

Αν η μεταγλώττιση σταματήσει με μήνυμα «Killed», το VM έμεινε από μνήμη. Ξανατρέξτε με λιγότερες παράλληλες εργασίες, π.χ. `make -j2`. Το make συνεχίζει από εκεί που σταμάτησε.

3.6 Εγκατάσταση

Πρώτα θα εγκαταστήσετε τα modules και μετά τον πυρήνα:

```
sudo make modules_install
sudo make install
```

Το `modules_install` αντιγράφει τα modules στο `/lib/modules/7.3.0-oslab/`. Το `install` αντιγράφει τον πυρήνα στο `/boot` ως `vmlinuz-7.3.0-oslab`, μαζί με το `System.map` και το config. Στη συνέχεια, δημιουργεί το initramfs και ενημερώνει το GRUB.

Ελέγξτε ότι υπάρχουν όλα τα αρχεία:

```
ls -l /boot/*7.3.0-oslab*
```

Πρέπει να βλέπετε τα αρχεία `vmlinuz-7.3.0-oslab`, `initrd.img-7.3.0-oslab`, `System.map-7.3.0-oslab` και `config-7.3.0-oslab`.

3.6.1 Εμφάνιση του μενού του GRUB

Το Ubuntu, και ορισμένες εγκαταστάσεις Debian, κρύβουν το μενού εκκίνησης. Για να μπορείτε να επιλέξετε πυρήνα (και να γυρίσετε στον παλιό αν ο νέος δεν εκκινεί), ορίστε στο `/etc/default/grub` τις παρακάτω γραμμές (αλλάξτε τις αν υπάρχουν, προσθέστε τις αν λείπουν):

```
GRUB_TIMEOUT_STYLE=menu
GRUB_TIMEOUT=10
```

και εφαρμόστε την αλλαγή:

```
sudo update-grub
```

3.7 Εκκίνηση με τον νέο πυρήνα

Επανεκκινήστε το VM για να φορτώσει τον νέο πυρήνα:

```
sudo reboot
```

Ο νεότερος πυρήνας είναι συνήθως η προεπιλογή. Αν δεν είναι, επιλέξτε στο GRUB «Advanced options for Debian/Ubuntu» και μετά την εγγραφή με `7.3.0-oslab`. Μετά την εκκίνηση:

```
uname -r
cat /proc/version
```

Η `uname -r` πρέπει να τυπώνει `7.3.0-oslab`. Αν ο νέος πυρήνας δεν εκκινεί, επανεκκινήστε, διαλέξτε τον παλιό από το «Advanced options» και δείτε την Ενότητα 3.9 για πιθανά σφάλματα.

3.8 Έλεγχος: ένα module για τον νέο πυρήνα

Στην άσκηση θα γράψετε έναν driver ως Linux kernel module. Πριν ξεκινήσετε, βεβαιωθείτε ότι μπορείτε να χτίσετε και να φορτώσετε ένα ελάχιστο module στον πυρήνα που μόλις φτιάξατε.

Τα modules μεταγλωττίζονται μέσω του συνδέσμου `/lib/modules/$(uname -r)/build`, που πρέπει να δείχνει στο δέντρο όπου χτίσατε τον πυρήνα:

```
ls -l /lib/modules/$(uname -r)/build
```

ΠΡΟΣΟΧΗ: Μη σβήσετε τον φάκελο `~/kernel/linux-7.3` μετά την εγκατάσταση. Χρειάζεται για κάθε module που θα χτίσετε, και για το module της άσκησης.

Σε έναν νέο φάκελο, π.χ. `~/hello`, δημιουργήστε το αρχείο `hello.c`:

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/utsname.h>

static int __init hello_init(void)
{
    pr_info("hello: loaded on %s\n", utsname()->release);
    return 0;
}

static void __exit hello_exit(void)
{
    pr_info("hello: unloaded\n");
}

module_init(hello_init);
module_exit(hello_exit);
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("OSLAB sanity-check module");
```

Στον ίδιο φάκελο δημιουργήστε το `Makefile`. Οι γραμμές κάτω από τα `all:` και `clean:` ξεκινούν με Tab, όχι με κενά. Αν αντιγράψετε το κείμενο από το PDF, τα Tabs θα έχουν γίνει κενά και το make θα αποτύχει με «missing separator».

```
obj-m := hello.o
KERNELDIR ?= /lib/modules/$(shell uname -r)/build

all:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) clean
```

Μεταγλώττιση, φόρτωση, έλεγχος, αφαίρεση:

```
make
modinfo ./hello.ko
sudo insmod hello.ko
sudo dmesg | tail -n 3
lsmod | grep hello
sudo rmmod hello
sudo dmesg | tail -n 2
```

Στο `dmesg` πρέπει να δείτε «hello: loaded on 7.3.0-oslab» και, μετά το `rmmod`, «hello: unloaded». Αν το `insmod` αποτύχει με «Invalid module format», το module χτίστηκε για άλλον πυρήνα από αυτόν που τρέχει: ελέγξτε την `uname -r` και τον σύνδεσμο `build`.

3.9 Συνήθη προβλήματα

`No rule to make target 'debian/canonical-certs.pem'` . Δεν εφαρμόσατε τις δύο πρώτες εντολές `scripts/config` της Ενότητας 3.4. Τρέξτε τις, μετά `make olddefconfig`, και συνεχίστε τη μεταγλώττιση.

Το make κάνει ερωτήσεις για επιλογές του config. Το `.config` άλλαξε μετά το τελευταίο `make olddefconfig` (ή `localmodconfig`). Τρέξτε `make olddefconfig` και ελέγξτε ότι η `make listnewconfig` δεν τυπώνει τίποτα.

Η μεταγλώττιση σταματά με «Killed» ή το VM «παγώνει». Έλλειψη μνήμης. Λιγότερες παράλληλες εργασίες (`make -j2`) ή περισσότερη μνήμη στο VM.

`No space left on device` Ελέγξτε με `df -h`. Συνήθως σημαίνει ότι δεν εφαρμόσατε το `localmodconfig` ή δεν απενεργοποιήσατε τα debug symbols. Μετά τη διόρθωση του config τρέξτε `make clean` για να ελευθερώσετε χώρο και ξεκινήστε τη μεταγλώττιση ξανά.

`Kernel panic: VFS: Unable to mount root fs` Συνήθως λείπει το initramfs ή του λείπει ο driver του δίσκου. Εκκινήστε με τον παλιό πυρήνα και τρέξτε:

```
sudo update-initramfs -c -k 7.3.0-oslab
sudo update-grub
```

Αν επιμένει, ο driver του ελεγκτή δίσκου δεν ήταν φορτωμένος όταν τρέξατε το `localmodconfig`: ξεκινήστε τη ρύθμιση από την αρχή χωρίς το βήμα της Ενότητας 3.4.2.

Ο νέος πυρήνας δεν εμφανίζεται στο GRUB. Τρέξτε `sudo update-grub` και ελέγξτε ότι το `vmlinuz-7.3.0-oslab` υπάρχει στο `/boot`.

Ο πυρήνας δεν φορτώνει καθόλου σε σύστημα UEFI. («bad shim signature» ή παρόμοιο). Το Secure Boot είναι ενεργό. Δείτε την Ενότητα 3.1.

3.10 Αφαίρεση του πυρήνα

Αν θέλετε να αφαιρέσετε τον πυρήνα που φτιάξατε, εκκινήστε πρώτα με τον πυρήνα της διανομής (ποτέ δεν σβήνουμε τον πυρήνα που τρέχει) και μετά:

```
sudo rm /boot/vmlinuz-7.3.0-oslab /boot/initrd.img-7.3.0-oslab \
        /boot/System.map-7.3.0-oslab /boot/config-7.3.0-oslab
sudo rm -r /lib/modules/7.3.0-oslab
sudo update-grub
```

3.11 Σύνοψη εντολών

Για όσους έχουν ήδη περάσει τη διαδικασία μία φορά:

```
sudo apt update
sudo apt install -y gcc make gawk bc bison flex libssl-dev \
                    libelf-dev dwarves libdw-dev
mkdir -p ~/kernel && cd ~/kernel
wget https://cdn.kernel.org/pub/linux/kernel/v7.x/linux-7.3.tar.xz
tar xf linux-7.3.tar.xz && cd linux-7.3
cp /boot/config-$(uname -r) .config
yes "" | make localmodconfig
scripts/config --set-str SYSTEM_TRUSTED_KEYS ""
scripts/config --set-str SYSTEM_REVOCATION_KEYS ""
scripts/config --disable DEBUG_INFO_DWARF_TOOLCHAIN_DEFAULT \
               --disable DEBUG_INFO_DWARF4 --disable DEBUG_INFO_DWARF5 \
               --enable DEBUG_INFO_NONE
scripts/config --set-str LOCALVERSION "-oslab"
make olddefconfig
make -j$(nproc)
sudo make modules_install
sudo make install
sudo reboot
uname -r
```

4 Βοηθητικός κώδικας και πρώτη φόρτωση του driver

Ο βοηθητικός κώδικας περιέχει τον σκελετό του driver Lunix:TNG (το module `lunix.ko`) και δύο βοηθητικά εργαλεία:

- `mk-lunix-devs.sh`: δημιουργεί τα special files `/dev/lunix0-batt`, `/dev/lunix0-temp`, `/dev/lunix0-light` κ.ο.κ., ένα για κάθε μέτρηση κάθε αισθητήρα.
- `lunix-attach`: «συνδέει» τη σειριακή θύρα με τον driver, ορίζοντας σε αυτήν τη line discipline του Lunix:TNG. Από εκεί και πέρα τα bytes που φτάνουν στη θύρα δεν πάνε πια σε όποιον τη διαβάζει με `read()`, αλλά στο module, που αναγνωρίζει τα πακέτα και ενημερώνει τις μετρήσεις των αισθητήρων.

4.1 Λήψη και μεταγλώττιση

Αφού κατεβάσετε τον βοηθητικό κώδικα απο τη σελίδα του μαθήματος στο helios αποσυμπιέστε τον και μεταγλωττίστε τον.

```
tar xzf lunix-tng-helpcode.tgz
cd lunix-tng-helpcode
make
```

Το make χτίζει το module μέσω του συνδέσμου `build` που είδατε στην Ενότητα 3.8, δηλαδή για τον πυρήνα που τρέχει. Μεταγλωττίζει επίσης το πρόγραμμα `lunix-attach`.

4.2 Φόρτωση του module και δημιουργία των συσκευών

```
sudo insmod ./lunix.ko
lsmod | grep lunix
sudo ./mk-lunix-devs.sh
ls -l /dev/lunix*
```

Στην έξοδο της `ls`, κοιτάξτε τον major και τον minor αριθμό κάθε αρχείου. Ο major είναι κοινός για όλα (είναι ο αριθμός με τον οποίο ο driver καταχωρείται στον πυρήνα), ενώ ο minor κωδικοποιεί τον αισθητήρα και τη μέτρηση.

4.3 Σύνδεση της σειριακής θύρας

Ανοίξτε ένα δεύτερο τερματικό και παρακολουθήστε τα μηνύματα του πυρήνα:

```
sudo dmesg -w
```

Στο πρώτο τερματικό:

```
sudo ./lunix-attach /dev/ttyS0
```

Χρησιμοποιήστε τη συσκευή που βρήκατε στην Ενότητα 2.3. Το `lunix-attach` μένει ανοιχτό στο προσκήνιο όσο θέλετε να είναι συνδεδεμένη η θύρα με τον driver. Με Ctrl-C η θύρα επιστρέφει στην κανονική line discipline. Ενώ τρέχει, στο τερματικό του `dmesg` πρέπει να εμφανίζονται μηνύματα από το module καθώς φτάνουν και αναλύονται τα πακέτα.

Σε αυτό το σημείο το περιβάλλον σας είναι έτοιμο. Το μέρος του driver που δίνει τις μετρήσεις στον χρήστη (το character device) είναι αυτό που θα υλοποιήσετε στην άσκηση, οπότε είναι αναμενόμενο ένα `cat /dev/lunix0-temp` να μην επιστρέφει ακόμη χρήσιμα δεδομένα.

4.4 Αφαίρεση του module

Κάθε φορά που αλλάζετε τον κώδικα, σταματάτε το `lunix-attach` με Ctrl-C και επαναλαμβάνετε τον κύκλο:

```
sudo rmmod lunix
make
sudo insmod ./lunix.ko
sudo ./lunix-attach /dev/ttyS0
```

Το `rmmod` αποτυγχάνει με «Module lunix is in use» όσο τρέχει το `lunix-attach` ή όσο κάποια διεργασία έχει ανοιχτό κάποιο `/dev/lunix*`. Τα special files στο `/dev` δεν χρειάζεται να ξαναδημιουργηθούν, εκτός αν κάνετε επανεκκίνηση.

ΠΡΟΣΟΧΗ: Ένα bug στον driver μπορεί να προκαλέσει oops ή να «παγώσει» το VM. Είναι αναμενόμενο στον προγραμματισμό πυρήνα. Μετά από oops, κάντε επανεκκίνηση πριν συνεχίσετε, γιατί ο πυρήνας μπορεί να έχει μείνει σε ασυνεπή κατάσταση και το module συνήθως δεν αφαιρείται.

Για απορίες: `cslab-assist@cslab.ece.ntua.gr`. Όταν αναφέρετε πρόβλημα, αναφέρετε τον hypervisor και τη διανομή σας, την έξοδο της `uname -r` και τα σχετικά μηνύματα του `dmesg`.